DenseChain: Faster Transaction Processing in Public Blockchains

Abstract

We present DENSECHAIN, a public blockchain based on Ethereum that increases transaction throughput without changing the Proof-of-Work consensus protocol. The chief insight behind DENSECHAIN is that while consensus controls the rate at which blocks are created, the number of transactions in each block is limited by I/O bottlenecks. Public blockchains such as Ethereum keep the number of transactions in each block low so that participating servers have enough time to process each block before a new block is created; by removing the I/O bottlenecks in transaction processing, DENSECHAIN allows processing more transactions in the same amount of time. DENSECHAIN makes two novel contributions: the DENSECHAIN architecture that removes I/O from the critical path of transaction processing, and the distributed, multi-versioned DSM-TREE data structure that stores system state. We evaluate DENSECHAIN using workloads based on public Ethereum traces (including smart contracts). We show that a single DENSECHAIN miner provides throughput of 27.4K transactions per second (27× higher than a single Ethereum miner); in a geo-distributed setting with four regions spread across three continents, DENSECHAIN throughput is 20K tps.

1 Introduction

Blockchains maintain the history of transactions as an immutable chain of blocks; each block has an ordered list of transactions, and is processed after its parent or previous block. Blockchains can be public, allowing untrusted servers to process transactions [1,3], or private, allowing only a few specific servers [19]. The decentralized nature, transparency, fault tolerance, and auditability of public blockchains have led to several applications in a wide range of domains like cryptocurrencies [1, 3, 21], games [29], and healthcare [43].

At a high level, public blockchains work in the following manner. Servers participating in the blockchain, termed miners, receive transactions from users. Miners execute the transactions and package them up into blocks. A consensus protocol, like proof-of-work (PoW) [34], decides the next block to be added to the blockchain. With PoW, miners release a new block at a regular cadence (*e.g.,* every 10–12 seconds in Ethereum [3]); the miner tries to fit all the transactions it can execute into each released block.

Problem: Low throughput. Public blockchains suffer from low transaction throughput. Two widely-used public blockchains, Bitcoin [1] and Ethereum, process only tens of transactions per second, limiting their applications [33, 63].

Prior works trace back the low throughput of blockchains to proof-of-work (PoW) consensus. PoW limits the block creation rate so that miners have enough time to receive and process the previous block before the next block is created [58]. This ensures that most of the miners are building on the same previous block, preventing forks in the blockchain. Researchers have proposed new consensus protocols [6, 30, 31, 42, 46] to increase transaction throughput.

Metric	No state	State: 10M	Ratio
Time taken to mine txs (s)	1047	6340	6× ↑
# of txs per block	2150	833	2.5× ↓
tx throughput (tx/s)	28.6	4.7	6× ↓

Table 1: **Impact of State on Blockchain Throughput**. The table shows the throughput of Ethereum with proof-of-work consensus in two scenarios when 30K transactions are mined using three miners. In the first scenario, there are no accounts on the blockchain. In the second scenario, 10M accounts have been added. Despite no other difference, tx throughput is 6× lower in the second scenario; we trace this to the I/O involved in processing transactions.

Insight. We observe that while PoW limits the block creation rate, it *does not* limit the block size (the number of transactions in each block). The block size is limited by the rate at which miners can process transactions (§2). Processing transactions involves executing a transaction and modifying system state accordingly; this becomes more expensive as the blockchain grows. Table 1 experimentally shows that when the number of accounts increases, block size reduces in Ethereum with PoW consensus (even among only three miners). The chief insight in this paper is that *if we could increase the rate at which transactions are processed, we could increase the block size safely, without modifying the PoW consensus protocol*.

How could we increase the block size safely? Miners will still release a block every 10–12 seconds after PoW; however, miners will be able to pack more transactions into each block due to faster processing. Typically, when we increase the block size, the time taken to transmit the block, and the time taken to process it will increase. When Ethereum previously increased the maximum block size [23], it was observed that the time taken to propagate the block increased marginally, while the time taken to process transactions saw significant increase [24]. Thus, if we can increase the rate at which transactions are processed, we can increase the block size. Since we do not change PoW consensus or the block creation rate, the safety properties of the blockchain hold. Transaction confirmation latency is unaffected since a transaction is confirmed

once around ten blocks have built on the block containing the transaction, and the block creation rate remains the same. The rate at which transactions are confirmed will increase since the block creation rate is the same, but there are more transactions in each block.

Approach: reducing I/O bottlenecks in tx processing. This work takes the novel approach of increasing tx throughput by tackling I/O bottlenecks in tx processing. Processing a transaction involves executing the transaction and verifying that the transaction is valid. In Ethereum, both execution and verification require reading and writing state stored in a Merkle tree [13, 44] on local storage. Tx processing is bottlenecked by I/O: for example, processing a single block of 100 transactions in Ethereum requires performing more than 10K random I/O operations (100× higher) and takes hundreds of milliseconds even on a datacenter-grade NVMe SSD. By reducing these I/O bottlenecks, this works accelerates tx processing; this in turn increases the number of transactions a miner can pack into each block, increasing throughput.

The I/O bottlenecks in Ethereum tx processing arise from two sources. First, the Merkle tree is serialized and stored in a RocksDB [12] key-value store. As a result, traversing the Merkle tree requires multiple, expensive RocksDB reads [52]. Second, Ethereum miners process transactions one by one in the critical path; this reduces the parallelism in the system and makes overall transaction throughput bottlenecked by I/O. These I/O bottlenecks are an intrinsic part of how Ethereum (and more generally, public blockchains that use Merkle trees) are designed [53]; merely upgrading to faster storage (or even holding all state in memory) will not resolve these problems.

This work. This paper presents DENSECHAIN, a new architecture for building public blockchains, that increases the overall throughput with faster transaction processing. DENSECHAIN tackles I/O bottlenecks by designing a custom storage solution for blockchain; at the heart of this customized storage is a novel data structure, the Distributed, Sharded Merkle Tree (DSM-TREE). The DSM-TREE stores data in a multi-versioned fashion across shards, and allows concurrent reads and writes. DSM-TREE eliminates the serialization inherent in using key-value stores like RocksDB. DENSECHAIN deconstructs miners into three entities: storage nodes that store the data, miners that process transactions, and I/O-Helpers that fetch data from the storage and provide it to miners. By allowing I/O-Helpers to prefetch data on behalf of the miners, I/O is removed from the critical path in tx processing; moreover, multiple I/O-Helpers can prefetch data at the same time.

Challenges. The DENSECHAIN architecture has to solve several challenges to be effective. For example, Ethereum considered using stateless clients [26] with separate storage nodes; proofs that validate that the data sent (termed witnesses) are included in the blocks. The size of these blocks that must be sent over the network made the scheme impractical [2, 10], and the proposal was abandoned [56]. DENSECHAIN handles this by co-designing the storage nodes and caches at the miners, minimizing the data sent over the network. A second challenge is prefetching data for Turing-complete smart contracts [55]. As smart contracts can execute arbitrary code, it is not straightforward for the I/O-Helpers to prefetch all the required data. DENSECHAIN solves this by having I/O-Helpers *speculatively pre-execute* the tx to obtain data and proofs. A third related challenge is that I/O-Helpers may submit stale proofs to the miners (miners may update storage after I/O-Helpers finish reading). DENSECHAIN handles this by having miners tolerate stale proofs whenever possible, via *witness revision*, allowing txs that would otherwise abort to execute.

Implementation. We implement the DENSECHAIN prototype by modifying Ethereum. We chose Ethereum as the implementation base and comparison point for two main reasons. First, Ethereum has been in operation as a public blockchain for nearly six years, presenting a large amount of data to test our assumptions. Second, Ethereum supports Turing-complete smart contracts, allowing the codification of complex decentralized applications.

Security and Trust. Ethereum has already increased the block size in the past [16], without triggering any negative events such as increased forks [18]. DENSECHAIN does the exact same thing, without changing the PoW consensus or block creation rate; as a result, DENSECHAIN inherits the security guarantees as Ethereum. DENSECHAIN introduces new storage and I/O-Helper entities; however, neither storage, or I/O-Helpers, or miners trust each other. All data exchanged between these nodes is authenticated using Merkle trees and can be verified. As a result, the new architecture of DENSECHAIN, while certainly increasing the complexity of the system, does not change the core security or trust assumptions.

Evaluation. To evaluate DENSECHAIN, we generate synthetic workloads that mirror transactions on Ethereum mainnet. We analyzed Ethereum transactions and observed that user accounts involved in transactions have a Zipf distribution: 90% of transactions involve the same 10% of user accounts. We observed that only 10–15% of Ethereum transactions involved smart contracts. Our workload generator faithfully reproduces these distributions. We evaluated DENSECHAIN using these workloads and observe that a single DENSECHAIN miner processes transactions 27× faster than an Ethereum miner; we provide a breakdown of the performance difference. When the DENSECHAIN miners are geo-distributed across three continents (thus incurring significant network latency), DENSECHAIN throughput reduces only by 20% compared to a single miner, achieving 20K transactions per second. Since DENSECHAIN has the same Proof-of-Work consensus and block creation rate as Ethereum, DENSECHAIN finalizes transactions 20× faster (with the same latency) as Ethereum.

In summary, this paper makes the following contributions:

- The novel DENSECHAIN architecture (§3)

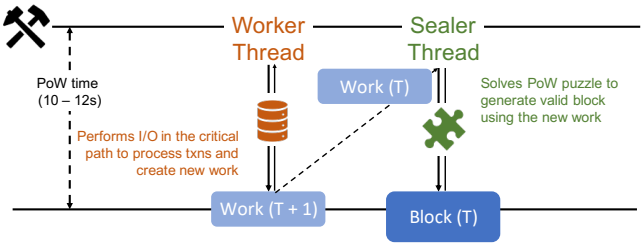


Figure 1: **How Ethereum miners work**. The worker thread processes txs, packages them into a block, and hands them to the sealer thread. The sealer thread takes 10–12 seconds to solve the PoW puzzle; the worker thread must process txs in this time-frame. I/O bottlenecks result in worker threads packing fewer txs into each block.

- The novel DSM-TREE authenticated data structure (§4)
- Empirical evidence that transaction throughput in public blockchains can be increased without modifying the Proof-of-Work consensus protocol (§6)

2 Background and Motivation

2.1 Public Blockchains and Ethereum

Blockchains are decentralized databases with support for transactions (txs). Blockchains log these txs as a chain of blocks; every block stores an ordered list of txs along with the cryptographic hash of its previous block (parent). Private blockchains allow specific servers to add to the blockchains, while public blockchains allow anyone to participate. As a result, all participating servers in public blockchains are untrusted; any of them can be malicious. Public blockchains depend on the non-malicious majority for correct behavior.

How do public blockchains work? We use Ethereum, a widely-used public blockchain, as an example. Servers that are trying to add a block to the blockchain are termed *miners*. Miners receive txs submitted by users, execute these txs, and group them into a block. We term executing txs and grouping into a block as *processing txs*. Several miners compete to add a block to the blockchain. Each miner tries to solve a Proof-of-Work (PoW) puzzle; if it solves the puzzle, it attaches the solution to the block and broadcasts it to other miners. Other miners verify the solution and begin to build on top of the received block. A tx is *confirmed* or *finalized* once ten blocks are built on top of the block containing the tx.

At a high level, a miner has two threads as shown in Figure 1. One worker thread is processing txs and grouping them into a block. The other sealer thread obtains the txs from the worker thread, and then tries to solve the PoW puzzle. The PoW consensus has miners emitting a block every 10–12 seconds, for example. While the sealer thread is working on one block, the worker thread tries to get the next block ready. Thus, the worker sealer has about 10–12 seconds to process txs; if it can process txs faster, it can pack more txs into the block it passes to the sealer thread.

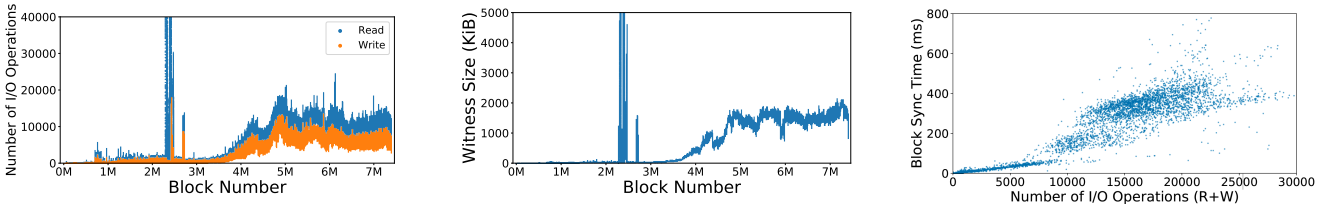
2.2 Problem: Low throughput

Ethereum and other public blockchains suffer from low throughput: only tens of txs added to the blockchain per second. The low throughput comes from two factors. First, the PoW consensus limits the block creation rate to one block every 10–12 seconds so that a majority of miners can receive and process a block before a new one is released; this ensures that every miner is building on the same previous block, preventing forks in the blockchain. Note that while PoW consensus limits the block creation rate, it doesn't directly limit the size of the block. The second factor lowering throughput is tx processing time. In each miner, the worker thread has to group txs into a block within 10–12 seconds; the rate at which the worker thread can process txs limits the maximum size of the block.

Tx processing. Processing a tx involves executing the tx and modifying system state such as account values. Since miners do not trust each other in a public blockchain, miners *authenticate data* and can prove that the data they provide is correct. This is done by maintaining a Merkle tree [44] over the data and publishing the latest Merkle root in the blockchain; another miner is able to independently verify that the data is correct, using the data and a vertical path in the Merkle tree (termed a *witness*).

Unfortunately, *accessing authenticated data becomes more expensive as the total size of the authenticated data increases* [62]. As a result, tx processing increasingly becomes bottlenecked on I/O. We demonstrate this with an experiment. We create two private Ethereum networks using the Geth client [15]; each network has three miners, 30K txs to mine, and the same proof-of-work (PoW) configurations. While the first network has only 3 miner accounts (total size: 220 MB), the second has 10M additional accounts (total size: 4 GB). Note that Ethereum currently has ≈130M accounts (total size: > 400 GB [14, 17]) in its blockchain state. Overall, the second network takes 6× more time and 2.5× more blocks to mine all txs using PoW consensus (Table 1). Using profilers, we see that in the second scenario, the time spent solving the PoW puzzle has not increased disproportionately; however, the worker thread takes 6× more time to process txs, and 69% of the time is spent in accessing and updating the system state. Thus, the miner's tx processing rate depends on the system state; this impacts the block size and overall throughput.

Empirical Study. We now study the I/O overheads observed on the Ethereum public blockchain. We use the Parity 2.2.11 client [9] to initialize a new server that joins the Ethereum network, and replays the blockchain (until 7.3 million blocks) to measure various costs. Since we want to observe the historical I/O cost, we do not mine new blocks; rather, we replay the blockchain and execute transactions in blocks that have



(a) Number of I/O operations *(b) Witness sizes* *(c) Block processing latency*

Figure 2: **I/O bottlenecks in processing Ethereum txs**. (a) shows how the number of I/O operations performed per block is increasing. (b) shows how the authenticated information (merkle witnesses) are increasing with increase in system state. (c) shows the increase in block processing time with the increasing number of I/O operations, highlighting the I/O bottleneck.

already been added, termed syncing. The I/O cost remains the same as when mining the block for the first time.

Reading Ethereum accounts results in multiple I/O operations. For processing a single block with around 100 transactions, Ethereum performs more than 10K random I/O operations (two orders of difference). Most of these I/O operations are performed for reading and updating user accounts in the Merkle tree. Figure 2(a) shows the number of I/O operations incurred in each block while processing txs, till 7.3 M blocks.

Data authentication causes significant I/O overhead. A witness is the vertical path in the Merkle tree required to verify a data item. Witness sizes represent the amount of data read and modified per block while reading and updating paths in the Merkle tree. In Ethereum, which uses secure 256-bit cryptographic hashes, the witness size of a single 100 byte user account (or value) can be above 4 KB, resulting in 40-60× overhead. The witness size also increases as the total data in the Ethereum state increases, as shown in Figure 2(b).

Tx processing is bottlenecked by I/O. We measure the time taken to process (or sync) each block: executing the txs in that block, and verifying if the resultant local Merkle root matches with the Merkle root in that block. Processing an Ethereum block with about 100 txs takes hundreds of milliseconds even on a datacenter-grade NVMe SSD. Figure 2 (c) shows the direct correlation between the time taken to process Ethereum blocks and the number of I/O operations performed, indicating that tx processing in Ethereum has I/O bottlenecks.

Summary. The I/O bottlenecks stem from storing the Merkle tree in an LSM-based [49] key-value store like RocksDB [12] that has inherent I/O amplification [51, 52]; Merkle nodes are indexed with their hash, randomizing node locations on disk.

2.3 Straw-man solutions

We consider some straw-man solutions for handling the I/O bottlenecks, and discuss why they are not appropriate.

Storing entire state in memory. Public blockchains such as Ethereum do not want to place restrictions on the hardware specifications of miners. If every miner is required to hold the entire authenticated state in memory, only machines with large amounts of DRAM would be able to participate. This breaks one of the central tenets of Ethereum that contributes to the decentralization of its miners.

Increase block size. Increasing the block size is the goal of this work; however, doing this in a naive fashion would not work. If we simply increased the number of txs in each block, miners receiving the block would need more time to process the block and build on top of it; as a result, the block creation rate would have to be lowered to ensure that previous block is processed by a majority of the miners before a new block is released. Overall, tx throughput would not increase though the block size increased. Ethereum has carefully increased the block size multiple times in the past [16]; it is prevented from raising the block size further due to I/O bottlenecks; with large blocks, new servers may take a long time to sync and participate in mining. This weakens decentralization. Thus, tackling the I/O bottleneck is crucial to increasing block size.

Alternative consensus protocols. Faster consensus protocols would result in blocks being released quicker, increasing overall throughput. The goal in this work is to increase throughput *without* changing the consensus protocol, and thus is orthogonal to work on new consensus protocols. Researchers have noticed that even faster consensus protocols ultimately run into the I/O bottlenecks in tx processing [65].

Thus, we need a mechanism to reduce the I/O bottlenecks in tx processing. DENSECHAIN achieves this goal with a new architecture and a novel authenticated data structure DSMTREE. With faster transaction processing, DENSECHAIN enables larger blocks *without* changing the PoW consensus protocol or the block creation rate.

3 DenseChain

DENSECHAIN is a public blockchain based on Ethereum that increases tx throughput without changing the Proof-of-Work (PoW) consensus or the block creation rate. DENSECHAIN minimizes the I/O bottlenecks in tx processing, thereby allowing miners to pack more txs into each block.

3.1 Overview

DENSECHAIN minimizes I/O bottlenecks using two techniques.

First, it makes each I/O operation cheaper by storing system state in the novel DSM-TREE data structure. In contrast to Ethereum, which accesses data from the RocksDB key-value store on SSDs (where each RocksDB get operation takes between a few hundred microseconds to a few milliseconds), DENSECHAIN stores the system state in memory using DSM-TREE. DSM-TREE is a sharded, multi-versioned in-memory authenticated data structure. Second, DENSECHAIN introduces a new architecture that removes I/O from the critical path. DENSECHAIN deconstructs miners into three entities: storage nodes, miners, and I/O-Helpers that read data from storage and submit to miners. In the common case, miners can process txs without performing I/O. Neither miners, nor storage nodes, nor I/O-Helpers trust each other: all data supplied by other entities is verified using Merkle witnesses before use.

DENSECHAIN differs from Ethereum in exactly two aspects: 1) miners are replaced by storage nodes, miners, and I/O-Helpers, and 2) the RocksDB-based storage is replaced with DSM-Tree. Everything else, such as the Proof-of-Work consensus, and the block creation rate, remains the same.

3.2 Building up the design step by step

In this section, we buildup the design of DENSECHAIN. We start with the problems that our study on Ethereum highlights. We discuss how DENSECHAIN solves these problems and the resulting challenges.

Problem-I: storing authenticated state in key-value stores leads to expensive I/O. Ethereum stores system state in a Merkle tree [13], which is stored in the RocksDB [12] key-value store. Traversing such a Merkle tree requires looking up nodes using their hashes. Hashing is computationally expensive and results in the nodes of the tree being distributed to random locations on storage. As a result, traversing the Merkle tree to read a leaf value requires several random read operations. The log-structured merge tree [47] that underlies RocksDB results in I/O amplification [51, 52].

Solution: store state in an optimized in-memory representation. DENSECHAIN introduces an in-memory version of the Merkle tree. Persisting the data is done via a write-ahead log and checkpoints. Traversing the Merkle tree is *decoupled* from hashing; obtaining the next node in the tree is a simple pointer dereference. Note that simply running RocksDB in memory would not be effective, as serializing and hashing Merkle nodes would still add significant overhead.

Resulting challenge: scalability and decentralization. As the blockchain grows, the amount of state in the Merkle tree will increase; soon, a single server's DRAM will not be sufficient. Furthermore, for maintaining decentralization, we cannot require miners participating in the blockchain to have significant amount of DRAM.

Solution: decouple storage from miners and shard the state. DENSECHAIN solves this problem using separate storage nodes, each of which is a commodity server. DENSECHAIN shards the Merkle tree into subtrees such that each subtree fits in the memory of a storage node. As the amount of data in the state increases, DENSECHAIN increases the number of shards. In this manner, DENSECHAIN scales with commodity miners and storage nodes without reducing the decentralization.

Problem-II: Miners perform I/O in the critical path. Transaction processing in Ethereum includes performing slow I/O operations in the critical path, and these transactions are processed one at a time.

Solution: decouple I/O and transaction execution. DENSECHAIN solves this problem by removing the burden of doing I/O from the miners. DENSECHAIN introduces I/O-Helpers that prefetch data and witnesses from the storage nodes and submit them to the miners. Miners use this information to execute transactions without performing I/O and asynchronously update the storage nodes. This architecture also increases parallelism as multiple I/O-Helpers can be prefetching data for different transactions at the same time.

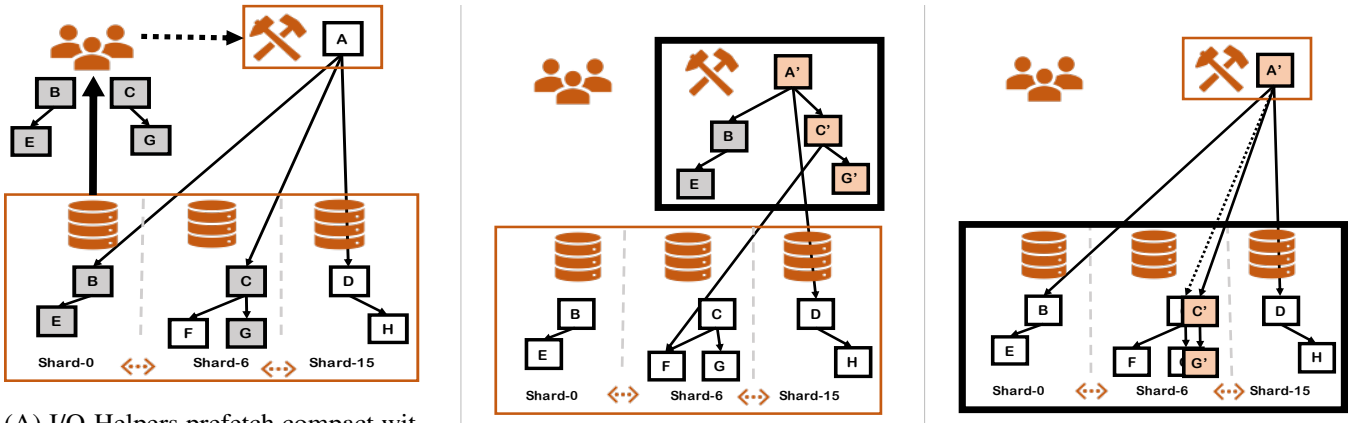
Resulting challenge: Prefetching I/O for smart contracts. One challenge with clients prefetching data for transactions is that some transactions invoke smart contracts. Smart contracts are Turing-complete programs that may execute arbitrary code. Thus, how does the client know what data to prefetch?

Solution: pre-execute transactions to get their read and write sets. DENSECHAIN solves this problem by having the clients pre-execute the txs. As part of this execution, the clients read data and witnesses from the storage nodes. One challenge is that the pre-execution may have different results than when the miner executes the tx (*e.g.,* the smart contract may execute different code based on the block which it appears in). We will describe how I/O-Helpers handle smart contracts correctly despite stale data from pre-execution (§3.4).

Resulting challenge: Consistency in the face of concurrency. Another challenge that arises from decoupling I/O from transaction execution is consistency. Multiple I/O-Helpers are reading from the storage nodes, and multiple miners are updating them in parallel. Using locks or other similar mechanisms will reduce concurrency and throughput.

Solution: using the two-layered, multi-versioned DSM-TREE. DENSECHAIN uses DSM-TREE to store the system state. The DSM-TREE has two layers: the *bottom layer* is sharded across the storage nodes and contains multiple versions. Every write causes a new version to be created in a copy-on-write manner; there are no in-place updates. As a result, concurrent updates from miners simply create new versions and do not conflict with each other. Each miner uses a DSM-TREE *top layer*: a private, writeable, consistent snapshot of the data.

Resulting challenge: DENSECHAIN has higher network traffic. Finally, the architecture of DENSECHAIN trades local disk I/O for remote network I/O. As a result, DENSECHAIN results in more network utilization, and the network bandwidth may become the bottleneck.



(A) I/O-Helpers prefetch compact witnesses

(B) Miners execute txns without I/O

(C) Shards update asynchronously

Figure 3: **DENSECHAIN architecture**. The figure shows how DENSECHAIN processes a tx that reads and updates accounts in two shards that are along the paths *ABE* and *ACG*. (A) I/O-Helpers prefetch compact witnesses *BE* and *CG* from storage nodes and submit to miners. (B) Miners verify and use these witnesses to execute tx against their top layer, and later update storage nodes. (C) Storage nodes verify updates from miners and asynchronously update their bottom layer, creating a new version for the modified account $A'C'G'$.

Solution: DENSECHAIN reduces network I/O via deduplication and the synergy between the DSM-TREE layers. DENSECHAIN uses multiple optimizations to reduce network I/O. First, the bottom and top layer of DSM-TREE collaborate with each other to reduce network traffic. Second, when any component of DENSECHAIN sends witnesses over the network, it will batch witnesses and perform deduplication to ensure only a single copy of each Merkle tree node is sent. Finally, miners send logical updates to storage nodes rather than physical updates as logical updates are smaller in size.

3.3 Architecture

DENSECHAIN introduces three kinds of participating entities: I/O-Helpers, miners, and storage nodes. Users send txs to I/O-Helpers, which pre-execute these txs and prefetch data and witnesses from storage nodes. Figure 3(a) shows how I/O-Helpers submit txs and the pre-fetched information to miners. Miners are responsible for creating new blocks of transactions and extending the blockchain; each miner maintains a private DSM-TREE top layer. Figure 3(b) shows how miners use the submitted information to execute these transactions without performing I/O. Finally, miners create a new block, gossip it to other miners, and update the storage nodes. Storage nodes are responsible for maintaining and serving the system state. They use the multi-versioned bottom layer of the DSM-TREE to provide consistent data to I/O-Helpers while handling concurrent updates from miners. Figure 3(c) shows how the miners asynchronously update the bottom layer of the DSM-TREE at the storage nodes.

3.4 Speculative Pre-Execution

I/O-Helpers read all the witnesses required for executing a tx from storage nodes. While this is straight-forward for simple txs, how do I/O-Helpers handle Turing-complete smart contracts that may access arbitrary locations? I/O-Helpers handle this by *speculatively* pre-executing the smart contract to obtain the read and write set.

However, smart contracts can use the timestamp, or block number of the block in which they appear, during their execution at the miner. These values are not known yet during their pre-execution at the I/O-Helpers. I/O-Helpers speculatively return an estimated value while pre-executing the contract.

Our analysis of Ethereum contracts shows that despite providing estimated values, I/O-Helpers still successfully prefetch the correct witnesses and node bags. For example, the CryptoKitties `mixGenes` function references the current block number and its hash. Since these numbers only affect written values (and not the read set), substituting approximate values does not affect the witnesses that are pre-fetched.

Stale data. We observe that I/O-Helpers can pre-execute with *stale* data and still prefetch the correct witnesses. For example, many contracts are *fixed-address* contracts, with behavior depending only on call inputs. To deal with rare *variable-address* contracts, the miner may asynchronously read from a storage node after the transaction is submitted. Even in these cases, the I/O-Helper will have retrieved some of the correct witnesses required for the transaction (*e.g.,* the `to` and `from` accounts).

3.5 Discussion

We note that DENSECHAIN differs from Ethereum in only two aspects: its architecture and its storage. Aspects such as Proof-of-Work consensus and block creation rate remain the same.

We discuss how this affects trust, incentives, and security and discuss the trade-offs.

Trust assumptions. DENSECHAIN does not require trust between any of its components. Miners operate without trusting I/O-Helpers or the storage nodes, as miners re-execute transactions and verify the data they receive from I/O-Helpers; I/O-Helpers verify the data they read from storage nodes; and storage nodes verify the block and its updates from miners.

Incentives. We sketch a possible incentive model here. A more rigorous analysis would require game-theoretic and economic models, which are beyond the scope of this paper. Users can prefetch data from the storage nodes themselves or pay I/O-Helpers. Users, I/O-Helpers, and miners pay the storage nodes for reading authenticated data. Miners get paid for mining a new block of transactions through block rewards. Every DENSECHAIN component can detect misbehaving entities and blacklist them, incentivizing correct behavior. The Ethereum stateless client proposal [26] with separate storage nodes received community support; hence, we are optimistic that the DENSECHAIN architecture can be incentivized.

Security. DENSECHAIN provides similar security guarantees as Ethereum, as it does not change the consensus protocol (Proof-of-Work) or trust assumptions between participating servers. DENSECHAIN does not impact the block creation rate, as it packs more transactions per block without changing the total time taken to process a block of transactions.

Trade-offs. DENSECHAIN introduces new storage nodes and I/O-Helpers; while this adds more complexity into the system, Ethereum was already considering adding storage nodes and stateless clients. Thus, we believe the additional complexity of DENSECHAIN is a good trade-off for its benefits. DENSECHAIN trades off local storage I/O for accessing memory over network, so the network may become a bottleneck. DENSECHAIN recognizes this risk and uses multiple techniques like utilizing the memory available at the miners to cache the top layer of the DSM-TREE, and performing witness compaction and deduplication to reduce network traffic. Although DENSECHAIN uses extra resources for separate storage nodes, storage nodes are shared across miners, amortizing the costs. I/O-Helpers also use extra resources; however, they are a performance optimization. If users judge I/O-Helpers not useful, users can prefetch from storage nodes, or turn off I/O-Helpers, causing miners to perform I/O; overall, correctness is not affected.

4 DSM-Tree

DENSECHAIN stores the system state in the DSM-TREE data structure. The Distributed, Sharded Merkle Tree (DSM-TREE) is an in-memory, multi-versioned, sharded, two-layer variant of the Merkle tree. We first present the common in-memory representation, then describe each of the two layers in turn, and then discuss how the layers collaborate and their trade-offs in different configurations.

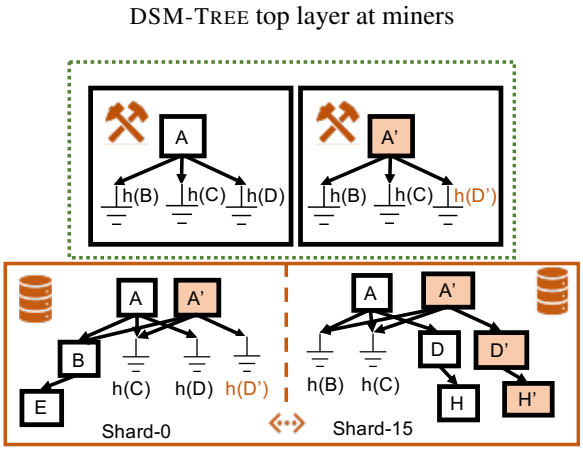


Figure 4: **DSM-TREE design in DENSECHAIN**. This figure shows the two-layered DSM-TREE where miners have their private copy of the top layer for consistency and the bottom layer is sharded for scalability. h(C) is a hash node for C.

In-Memory Representation. DSM-TREE builds the Merkle tree in memory using pointers. Tree traversal is decoupled from hashing: traversing the Merkle tree is done by dereferencing pointers; in contrast, Ethereum's Merkle tree has to perform expensive cryptographic hashing to find the next node during traversal. DSM-TREE uses periodic checkpoints for persisting the data. The checkpoints are only used to reconstruct the in-memory data structure in case of failures; reads are always served from memory.

Lazy Hash Resolution. When a leaf node in a Merkle tree is updated, hashes of nodes from the leaf to the root need to be recomputed. Recomputing hashes is expensive as nodes have to be serialized before being hashed. DSM-TREE defers doing this recomputation; on a write, only the leaf node is updated; when the leaf node is read, all the nodes from root to the leaf's parent are rehashed. Lazy hash resolution improves performance significantly by saving expensive hashing and serialization operations [11].

4.1 Bottom Layer

The bottom layer consists of a number of shards. Each shard is a vertical subtree of the Merkle tree, stored in DRAM. The bottom layer supports multiple versions to allow concurrent updates, as shown in Figure 4. The bottom layer has a write-ahead log to persist logical updates.

Multi-versioning. Each write to the bottom layer creates a new logical version of the tree, in a copy-on-write manner. There are no in-place updates. This versioning is required as miners may submit multiple blocks concurrently that potentially conflict with each other; the bottom layer creates a new version for each write. Thus, writes never conflict with each other, and DSM-TREE does not require locking or additional

coordination among miners or clients.

Garbage collection. Garbage collection of versions is driven by the higher-level blockchain semantics. When multiple miners are working on competing forks of the blockchain, multiple versions are maintained. Eventually, one of the forks is accepted as the mainline fork, and the others are discarded (and their associated versions are garbage collected by the bottom layer of the DSM-TREE).

4.2 Top Layer

Given that the bottom layer maintains multiple versions across multiple shards, what data should be read by the miners? To resolve this question, miners use the top layer. Each miner has a private top layer, that represents a consistent, writeable snapshot of the system state; it contains the first few levels of the Merkle tree, till a configurable retention level (r), and has the Merkle root node that summarizes a consistent snapshot of the entire system state. When a miner executes txs, all reads return values from this snapshot of the system.

As the miner executes transactions, their top layer is updated, switching to a different consistent view of the system state, as shown in Figure 4. The miner asynchronously updates the bottom layer's storage shards.

Caching and Pruning. The top layer acts as an in-memory cache of witnesses for the miner. By design, the top layer stores the recently used and the frequently changing parts of the Merkle tree. When the miner receives witness from I/O-Helpers, the top layer uses the witnesses to reconstruct a *partial* Merkle tree that allows miners to execute transactions, typically without performing I/O. The top layer also supports pruning the partial Merkle tree to help miners reclaim memory. Pruning replaces the nodes at the retention level ($r+1$) with *Hash nodes*. Hash nodes are placeholders that help miners to identify the DSM-TREE shard which has the pruned nodes.

Witness Revision. The bottom layer of the DSM-TREE is updated asynchronously by miners. As a result, the top layer (miner) may receive stale witnesses from the bottom layer (via I/O-Helpers). We introduce a new technique termed *witness revision* to tolerate stale witnesses. A witness is determined to be stale or incorrect because the Merkle root in the witness doesn't match the top layer's Merkle root. However, this could happen because of an *unrelated update* to another part of the Merkle tree. For example, the top layer might contain one vertical path; a different vertical path might have been updated. The top layer detects when this happens, and *revises* the witness to make it current by applying updates known to the miner to the witness. If the Merkle root matches now, then the witness is accepted. Witness revision is similar to doing `git push` (trying to upload your changes), finding out something else in the repository has changed, doing a `git pull` (obtaining the changes in the repository) to merge changes, and then doing a `git push`. With witness revision, the top layer tolerates stale data from the bottom layer and allows miners to execute non-conflicting transactions that would otherwise get rejected. Note that, witness revision cannot revise every potential stale witnesses. If the top layers are pruned aggressively, they may have insufficient information to detect if the changes are from an unrelated part of the Merkle tree.

4.3 Collaboration between the layers

The top and bottom layers collaborate to reduce network traffic. We also briefly discuss the potential DSM-TREE configurations with r (retention at the top layer) and c (compaction level at the bottom layer), and the tradeoffs involved.

Witness compaction. As the top layer of the DSM-TREE stores the top levels of the Merkle tree, the storage nodes do not need to send a full witness. Like the configurable retention level at the top layer r, the bottom layer has a configurable compaction level, c. Only nodes below the compaction level (compact witnesses) are sent in witnesses, after deduplicating nodes across witnesses, reducing the network burden of transmitting witnesses.

Configurations. The DSM-TREE can be configured to operate entirely from local memory without any network overhead, or just from remote memory with high network utilization. For example, if the top layer of the DSM-TREE has $r = \infty$, then the top layer caches the entire Merkle tree and is fully served from local memory. Similarly, if the bottom layer has $c = 0$, then un-compacted witnesses are sent over the network and accessed entirely from remote memory. These parameters can be tuned based on available memory and network capacity.

Tradeoffs. In a Merkle tree that has n levels, any DSM-TREE configuration that satisfies $c >= (n-r)$ allows the top layer to use compacted witnesses from the bottom layer. Note that having a higher r results in a lower number of transaction aborts, as the top layer has more information to detect non-conflicting updates and perform witness revision. The top layers should therefore set r based on the amount of memory available. Pruning the top layer should only be done under memory pressure.

4.4 Summary

The DSM-TREE is a novel variant of the Merkle tree modified for faster transaction processing in public blockchains. DSM-TREE presents a new point in the design space of authenticated data structures. The top layer exploits the cache-friendliness of the Merkle tree, while the sharded bottom layer relies on the fact that witness creation only requires a vertical slice of the tree. While the DSM-TREE is exclusively used with DENSECHAIN in this paper, it can be modified in a straightforward fashion to work with other blockchains.

5 Implementation

We implement DENSECHAIN and DSM-TREE in Typescript, targeting node.js. Miners and storage nodes use the DSM-TREE as a library. The performance critical portions of the code,

such as `secp256kp1` key functions for signing transactions and generating `keccak` hashes, are written as C++ node.js bindings. To execute smart contracts, we implement bindings for the Ethereum Virtual Machine Connector interface (EVMC) and use Hera (v0.2.2). Hera can run contracts implemented using Ethereum flavored WebAssembly (ewasm) or EVMC1 bytecode through transcompilation. The I/O-Helper is implemented in C++. The DSM-TREE and DENSECHAIN implementations, together 15K lines of code, will be open-sourced after acceptance. Our current implementation assumes storage is divided into 16 shards; this is configurable.

6 Evaluation

We seek to answer the following questions:

- What is the performance of a single miner? (§6.2)
- What is the end-to-end performance of DENSECHAIN in a geo-distributed setting? (§6.3)
- How is performance affected by tunable parameters? (§6.4)
- What are the overheads of DENSECHAIN? (§6.5)

The supplementary material contains more details for these experiments, along with additional experimental results.

6.1 Experimental Setup and Workloads

We run the experiments in a cloud environment on instances which are similar to the `m4.2xlarge` instance available on Amazon EC2 with 32GB of RAM and 48 threads per node. We use Ubuntu 18.04.02 LTS, and node.js v11.14.0. For the end-to-end benchmarks, each storage node, miner, and I/O-Helper is deployed on its own instance.

Workloads. We evaluate the performance of DENSECHAIN against synthetically generated workloads that mirror transactions on the Ethereum public *mainnet* blockchain. Since Ethereum transactions are signed, the public transactions cannot be used in experiments: we cannot change transaction data or the source accounts, because we do not have the `secp256k1` private key. To tackle this challenge, we analyze the public blockchain to extract salient features, and develop a *synthetic workload generator* which generates accounts with private keys we control so our clients can run and submit signed transactions.

Synthetic Workload Generator. We analyze the transactions in the Ethereum `mainnet` blockchain to build a synthetic workload generator. We analyzed 100K recent (since block 7M) and 100K older blocks (between blocks 4M and 5M) in the Ethereum blockchain to determine: 1) the distribution of accounts involved in transactions, 2) what fraction of all transactions are smart contract calls. We observe that 10-15% of Ethereum transactions are contract calls and the rest are simple transactions. This is true of both recent blocks and older blocks. It is also the case that a small percentage of accounts are involved in most of the transactions. Based the analyzed data, we generate workloads where 90% of accounts are called 10% of the time, and 10% of the accounts are called 90% of the time. Smart contracts are invoked 15% of the time.

System/optimization	`Get`	`Put`
In-memory Ethereum MPT	1x	1x
Pointer-based traversal	2.7×	2.3×
Batching and Lazy hash resolution	56×	69×
All optimizations	150×	160 ×

Table 2: **Performance breakdown**. The table shows the throughput of DSM-TREE (relative to in-memory Ethereum merkle tree) on gets and puts with different optimizations.

6.2 Performance of a single miner

The performance of a single miner depends on three things: the DSM-TREE data structure, the I/O-Helpers, and the top-layer cache at the miner. We first evaluate the performance of the DSM-TREE data structure, and then measure overall tx processing performance on a single node varying the number of I/O-Helpers. We then show the performance of the miner when varying the retention level of the top-layer cache.

DSM-TREE performance. For a fair comparison, we configure Ethereum to use an in-memory key-value store for storage. Ethereum uses the in-memory Merkle Patricia-Trie (MPT) [7] implemented using the `memdown` red-black tree [8]. We use `put` operations to recreate the system state corresponding to four million blocks on the Ethereum public chain. This results in 1.19M accounts. We read all accounts sequentially using `get` operations. `Put` operation creates or updates user account with 160-bit Ethereum address; `get` operation returns the RLP-encoded [11] Ethereum account at the address, along with its witness which has RLP-encoded Merkle nodes. We also compare the memory used.

Performance breakdown. DSM-TREE outperforms Ethereum's in-memory Merkle tree significantly in both gets and puts, as shown in Table 2. Note that both data structures are in memory, so the performance difference comes from other optimizations. First, Ethereum MPT performs RLP encoding and serialization/deserialization for reading and writing each node in the Merkle tree; DSM-TREE optimizes this with memoization and batching. Memoizing RLP-encoded nodes in DSM-TREE increases performance by 12× relative to Eth MPT. Batching `get` operations so that common nodes in the upper part of the Merkle Tree are only de-serialized once increases performance by 56× relative to MPT. Eth MPT has to hash nodes to traverse the Merkle Tree; in contrast, DSM-TREE uses pointers for constructing the tree, and eliminates hashing. This feature improves performance by another 2.7×, bringing the overall performance difference between DSM-TREE and Eth MPT to be 150×. A good

overall intuition for the performance difference is between a linked list in memory versus a linked list where each node is serialized and stored in an in-memory key-value store.

The performance difference for puts is similar (160×). The efficient memory representation of DSM-TREE contributes to 2.3× of this performance difference; the rest of the difference is due to lazy hash resolution. Lazy hash resolution defers recomputing the hashes of inner tree nodes until they are read. As a result, only the leaf nodes are updated and rehashed in the critical path. If we force all the Merkle nodes to be updated after every thousand updates, the performance difference with Eth MPT drops to 5× overall for puts.

Memory consumption. For the same system state with 1.19M accounts, DSM-TREE consumes 34× lower memory (775 MB) than Ethereum MPT (25 GB). This results from Eth MPT storing each node as a key-value pair. The reduced memory consumption of DSM-TREE is important since we want each shard to fit in the memory of a commodity server.

Overall performance. An Ethereum miner can process 1000 txs per second, if the entire system state is stored in an in-memory key-value store. We measured the performance of a single DENSECHAIN miner with a single I/O-Helper when the top-layer caching is disabled; the miner is accessing system state from remote in-memory DSM-TREE shards. In this setting, DENSECHAIN can process 2600 txs per second (2.6× higher than Ethereum).

Performance with multiple I/O-Helpers. When we increase the number of I/O-Helpers to four, parallel prefetching by the clients increases performance to 7700 txs per second (2.9× higher). Overall, an DENSECHAIN miner (with four I/O-Helpers) outperforms an Ethereum miner by 7.7×.

Performance with top layer enabled. When we configure the miner to use the DSM-TREE top layer while retaining the first seven levels of the Merkle tree ($r = 7$), the miner can process 27400 txs per second (3.5× higher than when top layer was disabled, and 27× higher than a single Ethereum miner).

6.3 End-to-End Geo-distributed Experiment

We run a geo-distributed experiment, with varying numbers of regions across three continents. Each region has four I/O-Helpers, one miner, and 16 storage nodes, caching eight levels of the DSM-TREE tree ($r = 8$).

DENSECHAIN in a single region achieves a throughput of 25000 tps; this is slightly reduced from the 27400 tps in the previous section since the storage nodes are being accessed over a wide-area network. When we scale to four regions, the throughput drops to 20000 txs per second, thus retaining 80% of the single-region performance. When we ran a workload consisting purely of smart contracts (`OmiseGO Token`), DENSECHAIN achieved 17900 txs per second. Table 3 captures the performance of DENSECHAIN in various settings, right from a single miner with one I/O-Helper to the geo-distributed experiment.

Optimizations	Config	Txs/s
Baseline	Ethereum, 1 miner	1K (1×)
DENSECHAIN	1 miner, 1 helper, r=0	2.6 K (2.6×)
Prefetch in parallel	1 miner; 4 helpers, r=0	7.7K (7×)
Top layer caching	1 miner; 4 helpers, r=7	27.4 K (27×)
Geo-distributed	4 miners; 16 helpers, r=7	20K (20×)

Table 3: **Performance breakdown**. The table shows the throughput of DENSECHAIN with different optimizations. All configs use 16 storage shards. Helpers indicate I/O-Helpers. While parallel prefetching increases DENSECHAIN throughput by 2.9× from 2.6K to 7.7K tps, the DSM-TREE top layer caching and witness compaction further increase throughput by 3.5×, from 7.7K to 27.4K tps.

Tx confirmation latency remains the same as in Ethereum, since DENSECHAIN and Ethereum share the same block creation and confirmation logic (confirmed after ten blocks build on block containing tx). Since more txs are added to the chain per second, more txs are confirmed per second as well. All these experiments use the same PoW consensus as in Ethereum, thus demonstrating that it is possible to increase tx throughput without changing the consensus protocol.

6.4 Perf impact of tunable parameters

We discuss the impact of varying two configuration parameters: the retention level r (number of Merkle tree levels stored at the top layer), and the compaction level c (levels of the Merkle tree sent over the network by the storage nodes). More details and graphs for these experiments can be found in the supplementary material.

Impact of tuning retention level. Increasing the retention level increases tx processing performance, but also increases the memory consumption. Pruning the cache to a certain retention level (r) reduces the memory consumed by the miner but results in larger witnesses being transmitted. For the Merkle tree constructed in the geo-distributed experiment described previously, miners caching till a tree depth of five (r = 5) consumes only 40% of the memory consumed by storing the full DSM-TREE in memory. As we increase the number of I/O-Helpers, the impact of higher r decreases. For example, in a single region in the geo-distributed experiment, there was no performance difference between $r = 6$ and $r = 7$ for four I/O-Helpers, but performance improved by 35% between $r = 6$ and $r = 7$ with two I/O-Helpers.

Retention Level and Tx Abort rate. If the top layer doesn't cache enough levels, stale witnesses cannot be revised, leading to tx aborts. We evaluate DENSECHAIN with 16 storage shards, 1 miner, and 4 clients, with various r and c configurations to measure the transaction abort rate. Increasing r

reduces the transaction abort rate. However, when there are a large number of accounts, the contention on Merkle tree nodes reduces, increasing the number of witnesses that can be revised and thus, reducing the abort rate for a fixed r. With 1M accounts and r=6, DENSECHAIN aborts less than two tps. In cases where the txs get aborted, I/O-Helpers or users themselves can fetch up-to-date witnesses from storage nodes and resubmit transactions to miners.

Tuning compaction level. A higher compaction level c increases the DSM-TREE shard throughput for I/O-Helpers (since it reduces the size of the witnesses transmitted over the network); with 10M accounts in state and $c = 0$, shards process accounts reads at 1.36K ops/sec and with $c = 6$, they process 9.4K get ops/sec, showing a 7× increase in throughput per storage shard. The compaction level should be tuned alongside the retention level, with $c >= (n - r)$ for a Merkle tree with n levels.

6.5 Overheads

DENSECHAIN has two main sources of overhead. First, it trades local storage I/O for network I/O, hence resulting in more network traffic. Second, it requires the participation of more commodity servers as I/O-Helpers and storage nodes. We discuss these overheads and how DENSECHAIN mitigates them.

Network bandwidth requirements. DENSECHAIN can be configured to produce blocks of a given size. For example, if the network can only handle 1 MB blocks, DENSECHAIN can be configured to produce blocks of this size. In our experiments, we did not constrain DENSECHAIN, and saw that DENSECHAIN can pack about 240K transactions into each block (480× higher than Ethereum) on average, with the same Proof-of-Work consensus and block creation time. DENSECHAIN blocks are 24MB in size, compared to the recent Ethereum blocks that are 40-60 KB. Our geo-distributed experiment used 24 MB blocks over the wide-area-network without running into performance problems.

A second source of network traffic is gossiping witnesses between miners. Using witness compaction, batching, and deduplication, DENSECHAIN reduces witness sizes by 95%, allowing miners to advertise witnesses with commodity network bandwidths. Our witnesses sent over the network were a few KB in size.

Additional resource usage. DENSECHAIN requires nodes to operate as I/O-Helpers and storage nodes. While storage nodes keep all state in memory, the state is sharded so that each shard fits in the DRAM of a commodity server. Since the storage nodes are shared among all miners, they do not significantly increase the overall cluster requirements. Similarly, I/O-Helpers can also be shared between miners.

7 Related Work

In this section, we place our work DENSECHAIN and DSM-TREE in the context of prior research.

Stateless Clients. The Stateless Clients [26] proposal seeks to insert witnesses into blocks, allowing miners to process a block without I/O. Despite active discussions [2, 4, 10], this proposal has not been implemented due to large witness sizes [56]; a single, simple transaction can have 4-6KB witness sizes, resulting in 40-60× the network overhead. In contrast, DSM-TREE reduces witness sizes by 95%; DENSECHAIN does not insert witnesses in blocks, and uses I/O-Helpers to reduce the I/O burden on miners.

Sharding. Sharding the blockchain into independent parallel chains that operate on subsets of state [36, 39, 41, 59, 60, 68] reduces I/O overheads; however, requires syncing the independent chains for consistency, is less resilient to failures or attacks [50, 54, 67], and require complex cross-shard transactions protocols. In contrast, DENSECHAIN does not shard the blockchain; the storage is sharded, but all miners add to a single chain. DENSECHAIN does not require locking or additional communication for executing transactions across multiple storage shards. Payment channels [5, 32, 35, 38, 40, 45] that offload work to side chains are complementary to our work.

Dynamic accumulators. Merkle trees belong to a general family of dynamic accumulators [22, 27]. Merkle trees, allow fast processing but, proofs grow with the underlying state. Constant-size dynamic accumulators based on RSA signatures [22, 27] have fixed size proofs but, have low processing rates; improving their performance is an ongoing effort [25]. DSM-TREE provides a practical solution to achieve high processing rates and small witness sizes. Recent work has proposed many new authenticated data structures [20, 28, 37, 52, 53, 57, 64, 66]. In contrast to these works, DSM-TREE scales Ethereum's Merkle Patricia trie [13] without changing its core structure, or how proofs are generated.

Transaction execution. DENSECHAIN adopts a design similar to Solar [69] and vCorfu [61], where transactions are executed based on data from sharded storage. DENSECHAIN modifies the design for decentralized applications and authenticated data structures. This allows DENSECHAIN to execute transactions on sharded state without requiring locking or additional coordination among miners. Similar to RAMCloud [48], the DSM-TREE design argues that large random-access data structures can get higher throughput and scalability when served from memory over the network.

8 Conclusion

We presented DENSECHAIN, a public blockchain architecture that increases tx throughput without changing the Proof-of-Work (PoW) consensus protocol. DENSECHAIN achieves this by tackling the I/O bottleneck in tx processing, allowing miners to pack more txs into each block. DENSECHAIN introduces a novel architecture that moves I/O off the critical path, and a new authenticated data structure that provides cheap access to state. DENSECHAIN will be made publicly available.

References

[1] Bitcoin. https://bitcoin.org/en/, 2019.

[2] Detailed analysis of stateless client witness size, and gains from batching and multi-state roots. https://ethresear.ch/t/detailed-analysis-of-stateless-client-\witness-size-and-gains-from-batching-and-\multi-state-roots/862, 2019.

[3] Ethereum. https://github.com/ethereum/, 2019.

[4] Ethereum improvement proposals repository. https://github.com/ethereum/EIPs, 2019.

[5] Fast, cheap, scalable token transfers for ethereum. https://raiden.network/, 2019.

[6] Hybrid casper ffg. https://github.com/ethereum/EIPs/blob/master/EIPS/eip-1011.md, 2019.

[7] Implementation of the modified merkle patricia tree as specified in the Ethereum's yellow paper. https://github.com/ethereumjs/merkle-patricia-tree, 2019.

[8] In-memory abstract-leveldown store for Node.js and browsers. https://github.com/Level/memdown, 2019.

[9] Parity ethereum 2.2.11-stable. https://github.com/paritytech/parity-ethereum/releases/tag/v2.2.11, 2019.

[10] A possible solution to stateless clients. https://ethresear.ch/t/a-possible-solution-to-stateless-clients/4094, 2019.

[11] Recursive Length Prefix Encoding. https://github.com/ethereum/wiki/wiki/RLP, 2019.

[12] RocksDB | A persistent key-value store. http://rocksdb.org, 2019.

[13] The modified Merkle Patricia tree. https://github.com/ethereum/wiki/wiki/Patricia-Tree, 2019.

[14] Full Node sync with Default Settings. https://etherscan.io/chartsync/chaindefault, 2020.

[15] Geth ethereum 1.9.25-stable. https://github.com/ethereum/go-ethereum/tree/v1.9.25, 2020.

[16] Ethereum average gas limit chart. https://etherscan.io/chart/gaslimit, 2021.

[17] Number of unique addresses in ethereum. https://etherscan.io/chart/address, 2021.

[18] Uncles per day. daily count of uncles generated by the ethereum network. https://www.etherchain.org/charts/unclesPerDay, 2021.

[19] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, page 30. ACM, 2018.

[20] Maurice Bailleu, Jörg Thalheim, Pramod Bhatotia, Christof Fetzer, Michio Honda, and Kapil Vaswani. {SPEICHER}: Securing lsm-based key-value stores using shielded execution. In *17th {USENIX} Conference on File and Storage Technologies ({FAST} 19)*, pages 173–190, 2019.

[21] BCNext. The nxt cryptocurrency. https://nxt.org, November, 2013.

[22] Josh Benaloh and Michael De Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Workshop on the Theory and Application of of Cryptographic Techniques*, pages 274–285. Springer, 1993.

[23] The block. Ethereum miners are increasing the network's gas limit by 25 = https://www.theblockcrypto.com/linked/69053/ethereum-miners-vote-for-25-gas-limit-increase,, June 20, 2020.

[24] bloXroute Labs. Increasing eth's gas limit: What we can safely do today. = https://ethresear.ch/t/increasing-eth-s-gas-limit-what-we-can-safely-do-today/8121,, October 2020.

[25] Dan Boneh, Benedikt Bünz, and Ben Fisch. Batching techniques for accumulators with applications to iops and stateless blockchains. In *Annual International Cryptology Conference*, pages 561–586. Springer, 2019.

[26] Vitalik Buterin. The Stateless Clients Concept. https://ethresear.ch/t/the-stateless-client-concept/172, 2017.

[27] Jan Camenisch and Anna Lysyanskaya. Dynamic accumulators and application to efficient revocation of anonymous credentials. In *Annual International Cryptology Conference*, pages 61–76. Springer, 2002.

[28] Alexander Chepurnoy, Charalampos Papamanthou, and Yupeng Zhang. Edrax: A cryptocurrency with stateless transaction validation. 2018.

[29] Tonya M Evans. Cryptokitties, cryptography, and copyright. *AIPLA QUARTERLY JOURNAL*, 47(2):219, 2019.

[30] Ittay Eyal, Adem Efe Gencer, Emin Gun Sirer, and Robbert Van Renesse. Bitcoin-ng: A scalable blockchain protocol. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 45–59, Santa Clara, CA, 2016. USENIX Association.

[31] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, pages 51–68, New York, NY, USA, 2017. ACM.

[32] Matthew Green and Ian Miers. Bolt: Anonymous payment channels for decentralized currencies. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 473–489, 2017.

[33] Zane Huffman. CryptoKitties is Clogging the Ethereum Network. https://themerkle.com/cryptokitties-is-clogging-the-ethereum-network/, 2019.

[34] Markus Jakobsson and Ari Juels. Proofs of work and bread pudding protocols. In *Communications and Multimedia Security*, 1999.

[35] Thaddeus Dryja Joseph Poon. The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf, 2019.

[36] Vitalik Buterin Joseph Poon. Plasma: Scalable autonomous smart contracts. https://plasma.io/plasma.pdf, 2019.

[37] Janakirama Kalidhindi, Alex Kazorian, Aneesh Khera, and Cibi Pari. Angela: A sparse, distributed, and highly concurrent merkle tree. 2018.

[38] Rami Khalil and Arthur Gervais. Revive: Rebalancing off-blockchain payment networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 439–453, 2017.

[39] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 583–598, May 2018.

[40] Marta Lokhava, Giuliano Losa, David Mazières, Graydon Hoare, Nicolas Barry, Eli Gafni, Jonathan Jove, Rafał Malinowsky, and Jed McCaleb. Fast and secure global payments with stellar. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 80–96, 2019.

[41] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. A secure sharding protocol for open blockchains. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, pages 17–30, New York, NY, USA, 2016. ACM.

[42] Loi Luu, Yaron Velner, Jason Teutsch, and Prateek Saxena. Smartpool: Practical decentralized pooled mining. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1409–1426, Vancouver, BC, 2017. USENIX Association.

[43] Thomas McGhin, Kim-Kwang Raymond Choo, Charles Zhechao Liu, and Debiao He. Blockchain in healthcare applications: Research challenges and opportunities. *Journal of Network and Computer Applications*, 135:62–75, 2019.

[44] Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.

[45] Andrew Miller, Iddo Bentov, Ranjit Kumaresan, and Patrick McCorry. Sprites: Payment channels that go faster than lightning. *CoRR abs/1702.05812*, 306, 2017.

[46] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of bft protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, pages 31–42, New York, NY, USA, 2016. ACM.

[47] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33(4):351–385, 1996.

[48] John K. Ousterhout, Arjun Gopalan, Ashish Gupta, Ankita Kejriwal, Collin Lee, Behnam Montazeri, Diego Ongaro, Seo Jin Park, Henry Qin, Mendel Rosenblum, Stephen M. Rumble, Ryan Stutsman, and Stephen Yang. The ramcloud storage system. *ACM Trans. Comput. Syst.*, 33(3):7:1–7:55, 2015.

[49] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33(4):351–385, 1996.

[50] Tayebeh Rajab, Mohammad Hossein Manshaei, Mohammad Dakhilalian, Murtuza Jadliwala, and Mohammad Ashiqur Rahman. On the feasibility of sybil attacks in shard-based permissionless blockchains. *arXiv preprint arXiv:2002.06531*, 2020.

[51] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. PebblesDB: Building Key-Value Stores using Fragmented Log-Structured Merge Trees.

In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP '17)*, Shanghai, China, October 2017.

[52] Pandian Raju, Soujanya Ponnapalli, Evan Kaminsky, Gilad Oved, Zachary Keener, Vijay Chidambaram, and Ittai Abraham. mLSM: Making Authenticated Storage Faster in Ethereum. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*, Boston, MA, 2018. USENIX Association.

[53] Leonid Reyzin, Dmitry Meshkov, Alexander Chepurnoy, and Sasha Ivanov. Improving authenticated dynamic dictionaries, with applications to cryptocurrencies. In *International Conference on Financial Cryptography and Data Security*, pages 376–392. Springer, 2017.

[54] Alberto Sonnino, Shehar Bano, Mustafa Al-Bassam, and George Danezis. Replay attacks and defenses against cross-shard consensus in sharded distributed ledgers. *arXiv preprint arXiv:1901.11218*, 2019.

[55] Nick Szabo. Smart contracts. *Unpublished manuscript*, 1994.

[56] Peter Szilagyi. Are stateless clients a dead end? https://www.reddit.com/r/ethereum/comments/e8ujfy/are_stateless_clients_a_dead_end/, December, 10, 2019.

[57] Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist, and Dmitry Khovratovich. Aggregatable subvector commitments for stateless cryptocurrencies. *IACR Cryptol. ePrint Arch.*, 2020:527, 2020.

[58] Vitalik Buterin. Toward a 12-second Block Time. https://blog.ethereum.org/2014/07/11/toward-a-12-second-block-time/, 2014.

[59] Marko Vukolić. The quest for scalable blockchain fabric: Proof-of-work vs. bft replication. In *International workshop on open problems in network security*, pages 112–125. Springer, 2015.

[60] Jiaping Wang and Hao Wang. Monoxide: Scale out blockchains with asynchronous consensus zones. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 95–112, Boston, MA, February 2019. USENIX Association.

[61] Michael Wei, Amy Tai, Christopher J Rossbach, Ittai Abraham, Maithem Munshed, Medhavi Dhawan, Jim Stabile, Udi Wieder, Scott Fritchie, Steven Swanson, et al. vcorfu: A cloud-scale object store on a shared log. In *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, pages 35–49, 2017.

[62] Jeffrey Wilcke. The Ethereum network is currently undergoing a DoS attack. https://ethereum.github.io/blog/2016/09/22/ethereum-network-currently-undergoing-\dos-attack/, 2016.

[63] JI Wong. Cryptokitties is causing ethereum network congestion (2017).

[64] Cheng Xu, Ce Zhang, and Jianliang Xu. vchain: Enabling verifiable boolean range queries over blockchain databases. In *Proceedings of the 2019 international conference on management of data*, pages 141–158, 2019.

[65] Lei Yang, Vivek Bagaria, Gerui Wang, Mohammad Alizadeh, David Tse, Giulia Fanti, and Pramod Viswanath. Prism: Scaling bitcoin by 10,000 x. *arXiv preprint arXiv:1909.11261*, 2019.

[66] Mingchao Yu, Saeid Sahraei, Songze Li, Salman Avestimehr, Sreeram Kannan, and Pramod Viswanath. Coded merkle tree: Solving data availability attacks in blockchains. *arXiv preprint arXiv:1910.01247*, 2019.

[67] Jusik Yun, Yunyeong Goh, and Jong-Moon Chung. Trust-based shard distribution scheme for fault-tolerant shard blockchain networks. *IEEE Access*, 7:135164–135175, 2019.

[68] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. Rapidchain: Scaling blockchain via full sharding. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, pages 931–948, New York, NY, USA, 2018. ACM.

[69] Tao Zhu, Zhuoyue Zhao, Feifei Li, Weining Qian, Aoying Zhou, Dong Xie, Ryan Stutsman, Haining Li, and Huiqi Hu. Solar: towards a shared-everything database on distributed log-structured storage. In *2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18)*, pages 795–807, 2018.